

Autonomous Driving Perception Module: Traffic Sign Detection and Recognition

Authors: Mohamed Wael, Omar Bakry, Mahmoud Idris

Project Motivation

In the field of autonomous vehicles, understanding and responding to road infrastructure is as critical as the physical act of driving itself. Traffic signs serve as the primary communication channel between road infrastructure and drivers, conveying vital instructions such as speed limits, warnings, mandatory actions, and prohibitions. For an autonomous vehicle to operate safely, efficiently, and legally, its perception system must detect, classify, and respond to these signs in real time under varying environmental conditions.

Building on foundational perception tasks like pedestrian detection, this project advances the autonomous vehicle pipeline by establishing a reliable Traffic Sign Recognition (TSR) system. Moving beyond pure simulation or cloud-reliant research, this project is driven by the practical motivation of edge deployment. The system is tailored to transition from software experimentation to an embedded artificial intelligence system capable of operating within the strict computational constraints of low-speed physical prototype vehicles.

Dataset Origin and Characteristics

The training and evaluation data utilized in this perception module originates from the publicly available benchmark **Car Detection Dataset** hosted on Kaggle.

The dataset is structured to replicate real-world operational challenges and includes the following characteristics:

- **Format:** The data consists of images alongside bounding box annotations strictly following the standardized YOLO normalization format, where each label is defined by its class ID and normalized coordinates (`class_id cx cy w h`) bounded between 0 and 1.
- **Contents:** It includes real-world images captured under varying lighting, weather conditions, distances, and partial occlusions to ensure model robustness.
- **Classes:** The dataset encompasses 16 distinct traffic sign categories, which feature multiple variants of Speed Limits (ranging from Speed Limit 20 to Speed Limit 120), Stop signs, Yield signs, Warning signs, Green Lights, and Red Lights.
- **Data Split:** The dataset is partitioned using a standard baseline ratio of 70% for training, 20% for validation, and 10% for testing. This arrangement ensures an independent evaluation of the model's ability to generalize to unseen driving scenarios.

YOLO Model Architecture and Justification

The object detection architecture selected for this system is **YOLOv8n (Nano)**. While larger variants (such as YOLOv8s or YOLOv8m) offer marginal accuracy gains on high-end hardware, they are computationally prohibitive for embedded systems.

YOLOv8n was chosen based on the following justifications:

- **Hardware Resource Constraints:** The ultimate deployment target is a Raspberry Pi 4 equipped with a 4-core ARM CPU and no dedicated graphics processing unit (GPU). The Nano variant generates a tiny compiled network file (~6MB weights), making full-scale real-time onboard inference feasible.
- **Speed and Accuracy Trade-off:** YOLOv8n delivers an exceptional balance of inference speed and structural regression accuracy, which is non-negotiable for safety-critical real-time applications.
- **Optimization Potential:** By training the model using transfer learning from pre-trained COCO weights at a high resolution (640x640) to maintain high recall on small distant objects, the network can later be exported to an optimized ONNX format at a reduced inference resolution (416x416). This specific configuration maximizes CPU efficiency, lowering latency to approximately 80–120ms and achieving a stable 8–12 frames per second (FPS) on the Raspberry Pi 4 CPU.

Summary of Completed Project Tasks

The implemented software pipeline completely fulfilled the core methodology required by the project specifications, yielding the following milestones:

1. **Data Preparation and Engineering:** The dataset folder layout was structured into explicit train, validation, and test subsets for both images and labels. The environmental configuration file ([data.yaml](#)) was dynamically remapped to absolute local directory paths to support hardware accelerated execution.
2. **Model Training:** The YOLOv8n network was successfully trained for 100 epochs using the mandated Stochastic Gradient Descent (SGD) optimizer with an initial learning rate of 0.01, a cosine learning rate scheduler, and an early stopping patience threshold set to 20 epochs.
3. **Performance Benchmarking:** The trained system achieved a high level of accuracy on the validation dataset, yielding a Precision of 0.9418, a Recall of 0.9488, and an overall mAP@0.5 score of 97.23%. This significantly surpasses the baseline document performance benchmark of 85.2% mAP@0.5.
4. **Optimization and Video Inference:** The final model weights were successfully optimized and exported to the ONNX framework for embedded CPU deployment. Real-time prediction was performed on the provided driving stream ([video.mp4](#)), generating annotated bounding boxes, class labels, and confidence metrics across all processed sequences.